IOT LAB RECORD – 18 EXPERIMENTS

Simple theory, procedure, programs, sample outputs and results

Important note: The programs are simple starting examples. Change GPIO pins, Wi-Fi details, broker address, API keys and sensor libraries according to your actual board and lab setup. Never publish real passwords or API keys. Some experiments require extra modules or a cloud account.

EXPERIMENT 1: Install and configure Arduino IDE, ESP32 and Raspberry Pi OS

Aim

To install the required development software and verify the ESP32/Raspberry Pi setup with a basic program.

Apparatus / Software Requirements

- Computer with internet connection
- ESP32 development board and USB cable
- Raspberry Pi with microSD card, power supply, keyboard/display (if available)
- Arduino IDE, ESP32 board package, Raspberry Pi Imager and Raspberry Pi OS

Theory (4 points)

1. Arduino IDE is used to write, compile and upload programs to microcontroller boards.
2. ESP32 support is added to Arduino IDE by installing the ESP32 board package.
3. Raspberry Pi OS is a Linux-based operating system used on Raspberry Pi computers.
4. A simple LED blink or Python print program confirms that the installation is working.

Procedure (5 points)

5. Install Arduino IDE on the computer.
6. Open Boards Manager, search for ESP32 and install the ESP32 platform.
7. Connect the ESP32 using a USB data cable; select the correct board and COM port.
8. Use Raspberry Pi Imager to write Raspberry Pi OS to the microSD card and boot the Pi.
9. Upload the blink program to ESP32, or run a basic Python print program on Raspberry Pi.

Program

```
// ESP32 LED blink (Arduino IDE)
const int LED_PIN = 2;
void setup() {
  pinMode(LED_PIN, OUTPUT);
  Serial.begin(115200);
}
void loop() {
  digitalWrite(LED_PIN, HIGH);
  Serial.println("LED ON");
  delay(1000);
  digitalWrite(LED_PIN, LOW);
  Serial.println("LED OFF");
  delay(1000);
}
```

Note: GPIO 2 is common on many ESP32 development boards, but check the pinout of your particular board. For Raspberry Pi, the OS installation is verified separately.

Sample Output

Arduino Serial Monitor:
LED ON
LED OFF
LED ON
LED OFF

Raspberry Pi terminal test:
$ python3 -c "print('Raspberry Pi setup OK')"
Raspberry Pi setup OK

Result

The Arduino IDE and ESP32 environment were configured successfully. The basic program verified the available platform.

EXPERIMENT 2: Publish ESP32 sensor data using MQTT

Aim

To publish sensor readings from ESP32 to an MQTT broker and receive them using a subscriber.

Apparatus / Software Requirements

- ESP32 board and sensor (for example, DHT11)
- Wi-Fi connection
- Computer with Mosquitto MQTT broker/client
- Arduino IDE and PubSubClient library

Theory (4 points)

10. MQTT is a lightweight messaging protocol used in IoT applications.
11. The MQTT broker receives messages from publishers and forwards them to subscribers.
12. ESP32 can publish sensor values to a topic through Wi-Fi.
13. Username/password and TLS can protect MQTT communication when configured on the broker.

Procedure (5 points)

14. Install Mosquitto broker and client tools on a computer or server.
15. Create a broker user and password; configure TLS certificates if supported and required.
16. Install the PubSubClient library in Arduino IDE.
17. Enter the Wi-Fi name, password, broker IP address and topic in the ESP32 program.
18. Run an MQTT subscriber on the computer and check the messages published by ESP32.

Program

```
// ESP32 MQTT publisher (install PubSubClient)
#include <WiFi.h>
#include <PubSubClient.h>
const char* ssid = "YOUR_WIFI";
const char* pass = "YOUR_PASSWORD";
const char* broker = "192.168.1.10";
WiFiClient wifi;
PubSubClient mqtt(wifi);
```

```
void setup() {
  Serial.begin(115200);
  WiFi.begin(ssid, pass);
  while (WiFi.status() != WL_CONNECTED) delay(500);
  mqtt.setServer(broker, 1883);
}
void loop() {
  if (!mqtt.connected()) mqtt.connect("ESP32Client");
  if (mqtt.connected()) {
    mqtt.publish("factory/sensor", "Temperature=28,Humidity=60");
    Serial.println("Message published");
  }
  delay(3000);
}
```

Note: Use port 1883 only for a trusted test network. For production, configure broker authentication and TLS (commonly port 8883) and use the appropriate certificate/client settings. This basic example demonstrates the publish flow, not TLS.

Sample Output

Subscriber command:
mosquitto_sub -h 192.168.1.10 -t factory/sensor

Received:
Temperature=28, Humidity=60
Temperature=28, Humidity=60

Result

ESP32 successfully connected to the MQTT broker and published sample sensor data for the subscriber.

EXPERIMENT 3: Temperature and humidity monitoring using ESP32 and DHT11

Aim

To interface DHT11, LED, push button and buzzer with ESP32 and generate an alert when temperature crosses a set value.

Apparatus / Software Requirements

- ESP32 board
- DHT11 sensor
- LED and 220-ohm resistor
- Push button
- Buzzer
- Breadboard and jumper wires
- Arduino IDE and DHT sensor library

Theory (4 points)

19. DHT11 measures temperature and relative humidity.
20. ESP32 reads the sensor values through a digital data pin.
21. An LED and buzzer can provide a local warning.
22. A push button can be used as a manual input or acknowledgement control.

Procedure (5 points)

23. Connect DHT11 VCC, GND and DATA to the ESP32.
24. Connect the LED through a resistor, buzzer and push button to separate GPIO pins.
25. Install the DHT sensor library and select DHT11 in the program.
26. Upload the program and open Serial Monitor at 115200 baud.
27. Observe temperature and humidity; check that LED and buzzer turn on above the set temperature. Press the button to silence the buzzer.

Program

```
// DHT11 + LED + button + buzzer
#include<DHT.h>
#define DHTPIN 4
#define DHTTYPE DHT11
DHT dht(DHTPIN, DHTTYPE);
const int LED=2, BUZZER=18, BUTTON=19;
void setup() {
  Serial.begin(115200); dht.begin();
  pinMode(LED, OUTPUT); pinMode(BUZZER, OUTPUT);
  pinMode(BUTTON, INPUT_PULLUP);
}
void loop() {
  float t=dht.readTemperature();
  float h=dht.readHumidity();
  if (isnan(t) || isnan(h)) {Serial.println("Sensor error"); delay(2000); return;}
  Serial.print("Temperature: "); Serial.print(t);
  Serial.print("°C  Humidity: "); Serial.print(h); Serial.println(" %");
  bool alert = t > 30;
  digitalWrite(LED, alert ? HIGH : LOW);
  digitalWrite(BUZZER, (alert && digitalRead(BUTTON)==HIGH) ? HIGH : LOW);
  delay(2000);
}
```

Note: Connect the DHT11 according to its module pin labels. Use a suitable driver/transistor if the buzzer draws more current than a GPIO can supply.

Sample Output

```
Temperature: 32.0 °C  Humidity: 65.0 %
Alert: LED ON, Buzzer ON
(When temperature is 28 °C: LED OFF, Buzzer OFF)
```

Result

Temperature and humidity were displayed, and the LED/buzzer alert operated according to the predefined temperature condition.

EXPERIMENT 4: Upload Raspberry Pi sensor readings to a cloud platform

Aim

To read environmental data using a sensor connected to Raspberry Pi and upload the readings to ThingSpeak.

Apparatus / Software Requirements

- Raspberry Pi with Raspberry Pi OS
- DHT11 sensor
- Breadboard and jumper wires
- Python 3, gpiozero or an appropriate sensor library

- Internet connection and ThingSpeak account/channel Write API Key

Theory (4 points)

28. Raspberry Pi is a single-board computer that can run Python programs.
29. A sensor measures physical values such as temperature and humidity.
30. Python can read sensor values through GPIO pins using suitable libraries.
31. ThingSpeak stores IoT readings in cloud channels and displays them as graphs.

Procedure (5 points)

32. Connect the DHT11 data pin to Raspberry Pi GPIO 4 and connect power and ground.
33. Install Python and the required DHT sensor library.
34. Create a ThingSpeak channel with temperature and humidity fields.
35. Copy the channel Write API Key into the Python program.
36. Run the program and check that new readings appear in the ThingSpeak channel.

Program

```
# Raspberry Pi Python example (Adafruit_DHT library)
import time
import requests
import Adafruit_DHT

SENSOR = Adafruit_DHT.DHT11
PIN = 4
API_KEY = "YOUR_THINGSPEAK_WRITE_KEY"

while True:
    humidity, temperature = Adafruit_DHT.read_retry(SENSOR, PIN)
    if temperature is not None and humidity is not None:
        print("Temperature:", temperature, "C")
        print("Humidity:", humidity, "%")
        url = "https://api.thingspeak.com/update"
        data = {"api_key": API_KEY, "field1": temperature,
                "field2": humidity}
        response = requests.get(url, params=data, timeout=10)
        print("Cloud entry:", response.text)
    else:
        print("Sensor read failed")
    time.sleep(20)
```

Note: Install a Raspberry Pi-compatible DHT library for your OS/Python version. ThingSpeak free channels have update-rate limits; follow the platform's current minimum update interval.

Sample Output

Temperature: 29 C
Humidity: 62 %
Cloud entry: 125
(An entry number is returned when the update succeeds.)

Result

Raspberry Pi collected sensor readings and uploaded them to the ThingSpeak cloud channel.

EXPERIMENT 5: Install and configure Arduino IDE, ESP32 and Raspberry Pi OS

Aim

To install the required development software and verify the ESP32/Raspberry Pi setup with a basic program.

Apparatus / Software Requirements

- Computer, ESP32 board and USB data cable
- Raspberry Pi and microSD card
- Arduino IDE, ESP32 board package and Raspberry Pi Imager

Theory (4 points)

37. Arduino IDE is used to write and upload embedded programs.
38. The ESP32 board package enables ESP32 boards in Arduino IDE.
39. Raspberry Pi OS provides the operating system for Raspberry Pi.
40. A basic blink or print program checks the setup.

Procedure (5 points)

41. Install Arduino IDE.
42. Install ESP32 support using Boards Manager.
43. Connect ESP32 and select the correct board and port.
44. Write Raspberry Pi OS to the microSD card and boot Raspberry Pi.
45. Upload the blink program and verify the output.

Program

```
// ESP32 LED blink (Arduino IDE)
const int LED_PIN = 2;
void setup() {
  pinMode(LED_PIN, OUTPUT);
  Serial.begin(115200);
}
void loop() {
  digitalWrite(LED_PIN, HIGH);
  Serial.println("LED ON");
  delay(1000);
  digitalWrite(LED_PIN, LOW);
  Serial.println("LED OFF");
  delay(1000);
}
```

Note: This is the same setup task as Experiment 1; the same basic verification program is used.

Sample Output

```
LED ON
LED OFF
LED ON
LED OFF

Raspberry Pi setup OK
```

Result

The development environment was installed and verified successfully.

EXPERIMENT 6: TinyML classification on ESP32 using TensorFlow Lite Micro

Aim

To run a lightweight trained machine-learning model on ESP32 and classify sample sensor data.

Apparatus / Software Requirements

- ESP32 board
- Computer with Arduino IDE
- TensorFlow Lite Micro library/runtime
- A trained and converted small model (.tflite or converted C array)
- Sample sensor data

Theory (4 points)

46. TinyML means running machine-learning models on small devices.
47. TensorFlow Lite Micro is designed to run small models on microcontrollers.
48. A trained model learns patterns from sample data.
49. The ESP32 performs inference locally and prints the predicted class.

Procedure (5 points)

50. Collect or prepare labelled sample sensor data.
51. Train a small classification model on a computer.
52. Convert and quantize the model for TensorFlow Lite Micro.
53. Include the model in the ESP32 project and allocate tensor memory.
54. Run sample input values and display the predicted category.

Program

```
// Simple demonstration of classification logic.
// Replace this threshold logic with a trained TFLite Micro model
// for a real TinyML deployment.
float vibration = 0.72;
void setup() {
  Serial.begin(115200);
}
void loop() {
  String label;
  if (vibration < 0.30) label = "NORMAL";
  else if (vibration < 0.70) label = "WARNING";
  else label = "ABNORMAL";
  Serial.print("Input vibration: ");
  Serial.println(vibration);
  Serial.print("Predicted class: ");
  Serial.println(label);
  delay(2000);
}
```

Note: Important: this short program demonstrates the classification flow only. It is not itself a TensorFlow Lite Micro model.

Sample Output

Input vibration: 0.72
Predicted class: ABNORMAL

Result

The ESP32 demonstration classified the sample input. A trained TensorFlow Lite Micro model can replace the threshold logic for actual TinyML inference.

EXPERIMENT 7: Object, human movement and gas detection using ESP32

Aim

To interface ultrasonic, PIR and gas sensors with ESP32 and display alerts based on their readings.

Apparatus / Software Requirements

- ESP32 board
- HC-SR04 ultrasonic sensor
- PIR motion sensor
- MQ-2 or suitable gas sensor module
- Jumper wires and breadboard
- Arduino IDE

Theory (4 points)

55. An ultrasonic sensor measures distance using sound waves.
56. A PIR sensor detects movement of warm objects such as people.
57. A gas sensor module detects changes in gas concentration.
58. ESP32 reads the sensor outputs and displays warning messages.

Procedure (5 points)

59. Connect the ultrasonic trigger and echo pins to ESP32 GPIO pins; use a voltage divider for a 5 V echo signal.
60. Connect the PIR output and gas sensor analog output to suitable ESP32 pins.
61. Upload the program and open Serial Monitor.
62. Move an object near the ultrasonic sensor and move in front of the PIR sensor.
63. Expose the gas sensor only to safe test conditions and observe the alert messages.

Program

```
// ESP32: ultrasonic + PIR + gas sensor
const int TRIG=5, ECHO=18, PIR=19, GAS=34;
void setup() {
  Serial.begin(115200);
  pinMode(TRIG, OUTPUT); pinMode(ECHO, INPUT);
  pinMode(PIR, INPUT);
}
void loop() {
  digitalWrite(TRIG, LOW); delayMicroseconds(2);
  digitalWrite(TRIG, HIGH); delayMicroseconds(10);
  digitalWrite(TRIG, LOW);
  long duration = pulseIn(ECHO, HIGH, 30000);
  float distance = duration * 0.0343 / 2;
  int motion = digitalRead(PIR);
  int gasValue = analogRead(GAS);
  Serial.print("Distance: "); Serial.print(distance); Serial.println(" cm");
  Serial.println(motion ? "Movement detected" : "No movement");
  Serial.print("Gas value: "); Serial.println(gasValue);
  if (distance > 0 && distance < 50) Serial.println("Object nearby!");
  if (motion) Serial.println("Human movement alert!");
```

```
  if (gasValue > 1800) Serial.println("Gas level alert!");
  delay(1000);
}
```

Note: Gas thresholds depend on sensor calibration and environment. MQ sensors require warm-up and are not certified life-safety gas alarms.

Sample Output

```
Distance: 32.5 cm
Object nearby!
Movement detected
Human movement alert!
Gas value: 2100
Gas level alert!
```

Result

The ESP32 read the ultrasonic, PIR and gas sensor values and displayed the corresponding alerts.

EXPERIMENT 8: LoRaWAN / NB-IoT sensor data transmission (simulation)

Aim

To demonstrate transmission and reception of sample field sensor data using a permitted communication simulation.

Apparatus / Software Requirements

- Computer with Python 3
- Python editor or terminal
- Sample soil moisture/temperature values
- Internet access (optional for online LoRaWAN simulator)

Theory (4 points)

64. LoRaWAN is a low-power wide-area network technology for long-range IoT communication.
65. NB-IoT is a cellular technology designed for low-power IoT devices.
66. A sensor node sends readings through a network to a remote monitoring application.
67. A simulation can demonstrate the data flow when physical radio hardware is unavailable.

Procedure (5 points)

68. Open a Python editor or terminal.
69. Create a sample sensor reading containing soil moisture and temperature.
70. Encode the readings as a message.
71. Simulate sending the message to a remote receiver.
72. Run the receiver and verify that the same values are displayed.

Program

```
# Simple permitted simulation of sender and receiver
import json

sensor_data = {
   "device": "Field_Node_01",
   "soil_moisture": 42,
   "temperature": 29
}
```

```
# Sender: prepare message
message = json.dumps(sensor_data)
print("Transmitted:", message)

# Receiver: decode received message
received = json.loads(message)
print("Received device:", received["device"])
print("Soil moisture:", received["soil_moisture"], "%")
print("Temperature:", received["temperature"], "C")
```

Note: This is a software simulation of the data exchange, not a live LoRaWAN or NB-IoT radio connection. A real deployment needs compatible radio hardware, gateway/network service or cellular NB-IoT service.

Sample Output

```
Transmitted: {"device": "Field_Node_01", "soil_moisture": 42, "temperature": 29}
Received device: Field_Node_01
Soil moisture: 42 %
Temperature: 29 C
```

Result

The simulation successfully demonstrated the sending and receiving of sample environmental sensor data.

EXPERIMENT 9: Servo gate control and DC motor control

Aim

To control a gate using a servo motor when an authorized signal is received and demonstrate DC motor control.

Apparatus / Software Requirements

- ESP32 board
- Servo motor
- DC motor and motor driver module (L298N or suitable driver)
- Authorized input switch/button
- External motor power supply
- Jumper wires and Arduino IDE

Theory (4 points)

73. A servo motor moves its shaft to a selected angle.
74. ESP32 generates a control signal to position the servo.
75. A DC motor rotates continuously when suitable voltage is applied.
76. A motor driver is required to control a DC motor safely from ESP32.

Procedure (5 points)

77. Connect the servo signal wire to an ESP32 GPIO and provide suitable power and ground.
78. Connect the DC motor to a motor driver, not directly to ESP32 GPIO.
79. Connect the driver control pins to ESP32 and use a separate motor supply.
80. Upload the program and activate the authorized input.
81. Observe the gate open, wait for the delay, and close; verify DC motor operation.

Program

```
// Servo gate + DC motor driver demonstration
#include<ESP32Servo.h>
```

```
Servo gate;
const int SERVO_PIN=18, AUTH=19;
const int IN1=25, IN2=26;
void setup() {
  Serial.begin(115200);
  pinMode(AUTH, INPUT_PULLUP);
  pinMode(IN1, OUTPUT); pinMode(IN2, OUTPUT);
  gate.attach(SERVO_PIN);
  gate.write(0);
}
void loop() {
  if (digitalRead(AUTH) == LOW) {
    Serial.println("Authorized: Gate opening");
    gate.write(90);
    digitalWrite(IN1, HIGH); digitalWrite(IN2, LOW);
    delay(3000);
    gate.write(0);
    digitalWrite(IN1, LOW); digitalWrite(IN2, LOW);
    Serial.println("Gate closed");
    delay(1000);
  }
}
```

Note: Use a separate suitable power supply for motors and connect grounds together. The motor driver must match the motor voltage/current.

Sample Output

```
Authorized: Gate opening
Gate closed
```

Result

The servo gate opened for the specified delay and returned to the closed position. DC motor control was demonstrated through a motor driver.

EXPERIMENT 10: TinyML classification on ESP32 using TensorFlow Lite Micro

Aim

To classify sample sensor data locally using a lightweight machine-learning model on ESP32.

Apparatus / Software Requirements

- ESP32 board
- Arduino IDE
- TensorFlow Lite Micro runtime
- Trained lightweight model and sample data

Theory (4 points)

82. TinyML runs machine-learning models on small embedded devices.
83. TensorFlow Lite Micro supports inference on microcontrollers.
84. A trained model maps input sensor values to predefined classes.
85. Local inference reduces the need to send every reading to the cloud.

Procedure (5 points)

86. Prepare labelled sensor data.
87. Train a small classification model.
88. Convert the model into a microcontroller-compatible format.
89. Load the model into an ESP32 project.
90. Provide sample readings and display the predicted class.

Program

```
// Basic classification-flow demonstration
float sensorValue = 0.15;
void setup() { Serial.begin(115200); }
void loop() {
  if (sensorValue < 0.30)
    Serial.println("Predicted class: NORMAL");
  else if (sensorValue < 0.70)
    Serial.println("Predicted class: WARNING");
  else
    Serial.println("Predicted class: ABNORMAL");
  delay(2000);
}
```

Note: This is a simple logic example; actual TinyML requires a trained model and TensorFlow Lite Micro inference code.

Sample Output

```
Predicted class: NORMAL
```

Result

The sample input was classified locally. The demonstration can be extended by loading a trained TensorFlow Lite Micro model.

EXPERIMENT 11: UART, SPI and I2C communication using ESP32

Aim

To demonstrate data transmission and reception using UART, SPI and I2C interfaces of ESP32.

Apparatus / Software Requirements

- ESP32 board
- Second ESP32 or compatible peripheral/module
- Jumper wires
- Arduino IDE
- UART, SPI and I2C libraries (built into Arduino framework)

Theory (4 points)

91. UART is a serial communication method using TX and RX lines.
92. SPI uses clock, data and chip-select lines for fast communication.
93. I2C uses SDA and SCL lines and device addresses.
94. ESP32 supports all three interfaces for communicating with peripheral devices.

Procedure (5 points)

95. Connect UART TX to the other device RX and RX to TX, with common ground.
96. Connect SPI SCK, MOSI, MISO and CS pins to the SPI peripheral.
97. Connect I2C SDA and SCL pins to the I2C device with suitable pull-ups.

98. Upload the test program and open Serial Monitor.
99. Transmit test values and verify the received data using the connected peripheral or logic analyser.

Program

```
// ESP32 UART output + I2C scan + SPI transfer example
#include<Wire.h>
#include<SPI.h>
void setup(){
  Serial.begin(115200);            // UART
  Wire.begin(21,22);               // SDA, SCL
  SPI.begin(18,19,23,5);           // SCK, MISO, MOSI, SS
  Serial.println("UART: Hello Peripheral");
  Serial.println("I2C scan:");
  for(byte a=1; a<127; a++){
    Wire.beginTransmission(a);
    if (Wire.endTransmission()==0){
      Serial.print("I2C device at 0x");
      Serial.println(a,HEX);
    }
  }
  digitalWrite(5, HIGH);
  pinMode(5, OUTPUT);
  digitalWrite(5, LOW);
  byte received = SPI.transfer(0x55);
  digitalWrite(5, HIGH);
  Serial.print("SPI received: ");
  Serial.println(received, HEX);
}
void loop() {}
```

Note: Connect a real SPI slave/peripheral to obtain meaningful returned data. The example assumes the selected pins are available on the ESP32 board.

Sample Output

```
UART: Hello Peripheral
I2C scan:
I2C device at 0x3C
SPI received: FF
(Example values; actual results depend on connected modules.)
```

Result

UART output, I2C device scanning and an SPI transfer were demonstrated. Received values depend on the connected peripheral.

EXPERIMENT 12: Wi-Fi and Bluetooth Low Energy communication using ESP32

Aim

To connect ESP32 to Wi-Fi and demonstrate BLE communication for exchanging a sample message.

Apparatus / Software Requirements

- ESP32 board with Wi-Fi and BLE support
- Wi-Fi router/hotspot
- Android phone with a BLE scanner app

- Arduino IDE and ESP32 BLE library

Theory (4 points)

100. Wi-Fi connects ESP32 to a wireless local network and internet services.
101. Bluetooth Low Energy is designed for low-power short-range communication.
102. A BLE peripheral advertises a service and characteristic.
103. A compatible phone can scan the ESP32 and read or receive sample data.

Procedure (5 points)

104. Enter the Wi-Fi SSID and password in the program.
105. Connect ESP32 to the computer and select the correct board and port.
106. Upload the program and check the Wi-Fi connection in Serial Monitor.
107. Use a BLE scanner app on a phone to find the ESP32 device.
108. Connect to the advertised device and read the sample characteristic.

Program

```
// Wi-Fi connection test
#include <WiFi.h>
const char* ssid = "YOUR_WIFI";
const char* password = "YOUR_PASSWORD";
void setup() {
  Serial.begin(115200);
  WiFi.begin(ssid, password);
  Serial.print("Connecting");
  while (WiFi.status() != WL_CONNECTED) {
    delay(500); Serial.print(".");
  }
  Serial.println("\nWi-Fi connected");
  Serial.println(WiFi.localIP());
}
void loop() {}
```

Note: The code above verifies Wi-Fi. To complete BLE data exchange, add a BLE GATT server/service using the ESP32 BLE library or NimBLE-Arduino and test its characteristic with a phone.

Sample Output

```
Connecting....
Wi-Fi connected
192.168.1.105

BLE scanner (when BLE service is enabled): ESP32 found
```

Result

The ESP32 Wi-Fi connection was verified. BLE device discovery and data exchange can be tested using a compatible BLE service and scanner.

EXPERIMENT 13: Local machine condition prediction using a lightweight ML model

Aim

To run a lightweight machine-learning model locally on ESP32 or Raspberry Pi and identify abnormal machine conditions.

Apparatus / Software Requirements

- ESP32 or Raspberry Pi
- Sample machine sensor data (vibration/temperature)
- Python 3 or Arduino IDE
- A trained lightweight classification model (for actual ML)

Theory (4 points)

109. Machine sensors collect values such as vibration and temperature.
110. Abnormal readings may indicate a machine fault.
111. A lightweight model can classify readings into normal and abnormal conditions.
112. Local prediction provides an alert without continuously sending sensor data to the cloud.

Procedure (5 points)

113. Collect sample normal and abnormal machine readings.
114. Prepare and train a lightweight classification model.
115. Deploy the model on ESP32 or Raspberry Pi.
116. Provide sample sensor values to the model locally.
117. Display the predicted condition and an alert if the machine is abnormal.

Program

```
# Simple Raspberry Pi/Python demonstration
# Replace threshold logic with a trained model for real ML.
temperature = 78
vibration = 0.82

if temperature > 70 or vibration > 0.70:
    condition = "ABNORMAL"
else:
    condition = "NORMAL"

print("Temperature:", temperature, "C")
print("Vibration:", vibration)
print("Predicted machine condition:", condition)
```

Note: This is a rule-based demonstration, not a trained ML model. For a true ML experiment, load a trained model and run its inference on the same sample data.

Sample Output

```
Temperature: 78 C
Vibration: 0.82
Predicted machine condition: ABNORMAL
```

Result

The sample machine readings were analysed locally and an abnormal condition was displayed.

EXPERIMENT 14: Wi-Fi and Bluetooth Low Energy communication using ESP32

Aim

To connect ESP32 to Wi-Fi and exchange sample data using Bluetooth Low Energy.

Apparatus / Software Requirements

- ESP32 board
- Wi-Fi router
- Smartphone with BLE scanner
- Arduino IDE and ESP32 BLE support

Theory (4 points)

118. Wi-Fi provides wireless network connectivity.
119. BLE supports short-range, low-power data exchange.
120. ESP32 can operate as a BLE peripheral.
121. A phone can discover the ESP32 and access its BLE service.

Procedure (5 points)

122. Connect ESP32 to the computer.
123. Enter Wi-Fi credentials and upload the Wi-Fi test program.
124. Check the assigned IP address in Serial Monitor.
125. Start a BLE scanner on the phone and scan for ESP32.
126. Connect to its BLE service and verify the sample data characteristic.

Program

```
// ESP32 Wi-Fi connection test
#include <WiFi.h>
const char* ssid = "YOUR_WIFI";
const char* password = "YOUR_PASSWORD";
void setup() {
  Serial.begin(115200);
  WiFi.begin(ssid, password);
  while (WiFi.status() != WL_CONNECTED) delay(500);
  Serial.println("Wi-Fi connected");
  Serial.println(WiFi.localIP());
}
void loop() {}
```

Note: Same experiment as Experiment 12. Add a BLE GATT service/characteristic to exchange actual BLE data.

Sample Output

```
Wi-Fi connected
192.168.1.105
BLE scanner: ESP32 device detected
```

Result

Wi-Fi connectivity was verified and the ESP32 was ready for BLE discovery and data exchange.

EXPERIMENT 15: UART, SPI and I2C communication using ESP32

Aim

To transmit and receive sample data using UART, SPI and I2C communication interfaces.

Apparatus / Software Requirements

- ESP32 board
- Compatible UART/SPI/I2C modules or second controller

- Jumper wires
- Arduino IDE

Theory (4 points)

127. UART communicates using transmit and receive pins.
128. SPI transfers data using clock and separate data lines.
129. I2C connects devices using SDA and SCL lines.
130. These interfaces allow ESP32 to communicate with sensors and modules.

Procedure (5 points)

131. Wire UART TX/RX pins between the controller and peripheral.
132. Wire SPI clock, MOSI, MISO and chip-select pins.
133. Wire I2C SDA and SCL pins and ensure common ground.
134. Upload the communication test program.
135. Check the serial output and verify received values from the connected modules.

Program

```
// Basic I2C scan and UART message
#include<Wire.h>
void setup(){
  Serial.begin(115200);
  Wire.begin(21,22);
  Serial.println("UART message sent");
  Serial.println("Scanning I2C devices...");
  for(byte a=1;a<127;a++){
    Wire.beginTransmission(a);
    if(Wire.endTransmission()==0){
      Serial.print("Device found at 0x");
      Serial.println(a,HEX);
    }
  }
}
void loop(){}
```

Note: Same communication topic as Experiment 11. Use a suitable SPI slave/module to verify actual SPI receive data.

Sample Output

```
UART message sent
Scanning I2C devices...
Device found at 0x3C
```

Result

UART communication output and I2C device detection were verified. SPI can be tested with a connected SPI peripheral.

EXPERIMENT 16: Wi-Fi and Bluetooth Low Energy communication using ESP32

Aim

To configure ESP32 Wi-Fi and demonstrate BLE-based wireless communication.

Apparatus / Software Requirements

- ESP32 board
- Wi-Fi network
- Smartphone with BLE scanner
- Arduino IDE

Theory (4 points)

136. Wi-Fi enables ESP32 to join a wireless network.
137. BLE allows nearby devices to exchange small amounts of data.
138. ESP32 supports both Wi-Fi and BLE.
139. BLE services and characteristics define the data exchanged with a phone.

Procedure (5 points)

140. Connect the ESP32 to the computer using a USB cable.
141. Set Wi-Fi credentials in the program.
142. Upload the code and confirm the Wi-Fi connection.
143. Scan for the ESP32 using a BLE scanner app.
144. Connect to the BLE service and read or write a sample characteristic.

Program

```
//Wi-Fitestprogram
#include<WiFi.h>
constchar*ssid="YOUR_WIFI";
constchar*password="YOUR_PASSWORD";
voidsetup(){
 Serial.begin(115200);
 WiFi.begin(ssid,password);
 while(WiFi.status()!=WL_CONNECTED){
  delay(500);
  Serial.print(".");
 }
 Serial.println("\nConnectedtoWi-Fi");
}
voidloop(){}
```

Note: Same experiment as Experiment 12. The displayed code tests Wi-Fi; use a BLE GATT service for actual BLE message exchange.

Sample Output

```
ConnectedtoWi-Fi
BLEscan:ESP32devicedetected
```

Result

The ESP32 connected to Wi-Fi. BLE discovery can be used to establish short-range communication with a compatible device.

EXPERIMENT 17: UART, SPI and I2C communication using ESP32

Aim

To implement and verify serial communication using UART, SPI and I2C interfaces.

Apparatus / Software Requirements

- ESP32 development board
- Peripheral modules supporting UART, SPI and I2C
- Jumper wires
- Arduino IDE

Theory (4 points)

145. UART is an asynchronous serial protocol.
146. SPI is a synchronous protocol suitable for fast peripheral communication.
147. I2C supports multiple addressed devices on a shared bus.
148. ESP32 provides hardware support for these communication protocols.

Procedure (5 points)

149. Connect the ESP32 UART pins to a UART peripheral.
150. Connect the SPI pins to an SPI module.
151. Connect I2C SDA and SCL to an I2C sensor or display.
152. Upload the test code and start Serial Monitor.
153. Send test data and verify the response from each connected interface.

Program

```
// UART + I2C device scan
#include<Wire.h>
void setup(){
 Serial.begin(115200);
 Wire.begin(21,22);
 Serial.println("UART test message");
 for(int address=1; address<127; address++){
  Wire.beginTransmission(address);
  if (Wire.endTransmission()==0){
   Serial.print("I2C device: 0x");
   Serial.println(address, HEX);
  }
 }
}
void loop(){}
```

Note: Same topic as Experiments 11 and 15. SPI data verification requires a connected SPI peripheral.

Sample Output

```
UART test message
I2C device: 0x3C
```

Result

The communication test program ran successfully and detected the connected I2C device. UART output was also verified.

EXPERIMENT 18: Wi-Fi and Bluetooth Low Energy communication using ESP32

Aim

To connect ESP32 to Wi-Fi and exchange sample sensor readings or commands using BLE.

Apparatus / Software Requirements

- ESP32 board
- Wi-Fi router or hotspot
- Smartphone with BLE scanner app
- Arduino IDE and BLE library

Theory (4 points)

154. Wi-Fi connects the ESP32 to a wireless network.
155. BLE is used for short-distance wireless communication.
156. ESP32 can advertise a BLE device name and provide data characteristics.
157. A mobile device can connect and exchange data with the ESP32.

Procedure (5 points)

158. Connect ESP32 to the computer and open Arduino IDE.
159. Enter the Wi-Fi network details in the program.
160. Upload the program and verify the connection in Serial Monitor.
161. Open a BLE scanner on the phone and find the ESP32.
162. Connect to the BLE service and exchange a sample reading or command.

Program

```
// ESP32 Wi-Fi connection example
#include<WiFi.h>
const char* ssid = "YOUR_WIFI";
const char* password = "YOUR_PASSWORD";
void setup() {
  Serial.begin(115200);
  WiFi.begin(ssid, password);
  while (WiFi.status() != WL_CONNECTED) delay(500);
  Serial.println("Wi-Fi connected successfully");
  Serial.print("IP address: ");
  Serial.println(WiFi.localIP());
}
void loop() {}
```

Note: Same experiment as Experiments 12, 14 and 16. A BLE GATT service/characteristic is required for actual BLE data exchange.

Sample Output

```
Wi-Fi connected successfully
IP address: 192.168.1.105
BLE scanner: ESP32 detected
```

Result

The ESP32 Wi-Fi connection was established and the device was ready for BLE communication with a compatible device.